NEXT ROLE - FIND WORK THAT FITS

AI-Powered Resume Analysis and Job Role Recommendation System

B.Tech Project Synopsis

1. Introduction

1.1 Overview

Next Role is a full-stack web application that reads a candidate resume - in PDF, Word, plain-text, or even scanned-image form - and produces a ranked list of job roles that genuinely fit the candidate skills. For every matched role the system reports a match percentage, a plain-English explanation of why the role fits, and a required-skills checklist showing which skills the candidate already has and which are still to learn. The field of the project is software engineering, specifically web-based career information systems that combine resume mining with skill-based job matching.

1.2 Technology Used

The client is a single-page application built with React 18, TypeScript and Tailwind CSS, bundled by Vite. The server is a REST API written in Node.js with the Express framework. MongoDB is used for persistence when a connection string is provided; otherwise the application transparently falls back to a structured file store, so it never fails because of missing infrastructure. Analysis is powered in two tiers: the Grok large-language-model API from xAI (OpenAI-compatible chat-completions endpoint) produces AI-written summaries when an API key is configured, while a deterministic local matching engine keeps the product fully functional with zero external keys. Authentication supports email and password (scrypt hashing, JSON Web Tokens) and Google OAuth. Resume text is extracted by a three-stage pipeline: pdf-parse, pdf.js 3.x, and Tesseract OCR for scanned pages, with Mammoth handling Word documents. The desktop edition is packaged with Electron into a one-click Windows installer, and a browser-only edition is hosted statically on GitHub Pages.

1.3 Special Technical Terms

Applicant Tracking System (ATS): software employers use to parse and rank resumes automatically.

Skill extraction: identifying concrete, catalogue-matched technical and soft skills from unstructured resume text.

Skill-role ontology: the curated graph in nextrole.json linking 163 skills to 30 job roles and their required-skill lists.

Alias normalisation: mapping real-world spellings (e.g. "Node JS", "k8s", "postgres") onto one canonical skill.

OCR (Optical Character Recognition): converting scanned page images into machine-readable text.

Match percentage: normalised overlap score between a candidate skill set and a role required-skill set.

JWT (JSON Web Token) and scrypt: stateless session tokens and a memory-hard password hashing function.

SPA / static deployment: single-page client served without server-side routing, using hash-based navigation.

1.4 Objectives of the Project

Accept a resume upload or pasted text with explicit validation errors for file type, size and empty files.

Extract readable text from standard PDFs, modern PDFs, Word documents and scanned pages via OCR.

Rank at least six job roles against the candidate skills, best fit first, with match percentages and reasons.

Present a searchable 30-role library and per-role required-skill checklists with coverage counts.

Provide sign up / sign in (email + password, optional Google), and per-user saved analysis history.

Expose the same product as a local full-stack app, a hosted browser-only app, and a desktop application.

2. Feasibility Study

2.1 Operational Feasibility

The interface is deliberately simple: three sections - Home, Explore roles and Your profile - with a guided upload card, immediate results, and clear error messages written in plain language. No training is required for first-time users, accounts are optional for analysis, and match percentages are shown only after a real analysis so results are never misleading. The product therefore fits the work habits of students and early-career candidates with minimal organisational resistance.

2.2 Technical Feasibility

Every component of the proposed system is a mature, widely adopted open-source technology (React, Express, MongoDB, Tailwind CSS, Electron, pdf.js, Tesseract). The prototype has already been built and validated end-to-end: a scanned resume was recovered with OCR (3,101 characters, 15 skills) and matched to roles with sensible percentages. Degradation paths - local matching when the Grok key is absent, file store when MongoDB is absent, browser-only analysis when no server exists - remove single points of failure.

2.3 Economic Feasibility

Development and execution require no licence fees: the complete toolchain is free, hosting on GitHub Pages is free, and the desktop installer is self-contained. The only optional cost is a pay-as-you-go xAI API key for AI-written summaries, which is not required for core functionality. Hardware needs are modest and already available on any typical development machine.

2.4 Need and Significance of the Project

Job portals match candidates by titles and keywords rather than demonstrable skills, and a large share of real resumes arrive as scans or photographs that machines cannot read. Career switchers in particular possess the skills but lack the vocabulary of the target job description. Next Role addresses this gap by translating any resume into an explicit skill-role map the candidate can act on: what fits today, what is one course away, and which roles to apply for first. The system saves candidate time, gives recruiters pre-validated shortlists, and can be extended to college placement cells and recruitment portals.

3. Methodology and Planning of Work

3.1 Development Model

The project follows an iterative agile software-development life cycle: requirements were gathered and frozen as a single data contract (nextrole.json), then the system was built in short iterations - extraction, matching, accounts, interface, deployment - each ending with a working increment and a test pass. Requirements, design, code and documentation all reference the same data file, keeping the three tiers of the system consistent.

3.2 Phase 1 - Requirements and Data Modelling

All copy, the 163-skill catalogue, the 30-role library, required-skill lists, navigation labels and interface text are modelled once in nextrole.json and consumed by both client and server. This single source of truth simplifies maintenance, enables offline operation, and guarantees that the web, hosted and desktop editions always show identical content.

3.3 Phase 2 - Resume Extraction Pipeline

Text extraction runs as an ordered pipeline with fallbacks: pdf-parse for standard PDFs; pdf.js 3.x when the first engine cannot read the file; and finally Tesseract OCR, which renders each page to an image at 2.6x scale and reads it (up to eight pages) for scanned or photographed documents. Word files are converted with Mammoth and plain text is accepted as-is. Failures at every stage raise specific, actionable error messages (unsupported type, oversized file, empty file, password-protected PDF) instead of failing silently.

3.4 Phase 3 - Matching Engine and Grok Integration

The local matcher normalises skill aliases, matches separator-agnostic phrases, detects title hints (for example "front end developer" or "DevOps") and infers years of experience; it scores candidate skills against the skill-role graph, applies title-hint tie-breaks, guarantees at least six suggested roles even for thin resumes, and outputs match percentages with generated explanations. When GROK_API_KEY is set, the server first calls the xAI chat-completions API with the resume text and catalogue constraints, parses the structured JSON reply, and falls back to the local matcher on any error - the local engine is the safety net that keeps the product honest.

3.5 Phase 4 - Accounts, Security and Persistence

Passwords are hashed with salted scrypt and verified in constant time; sessions are signed JWTs carrying a server-generated identifier. The API handles duplicate sign-up (409), wrong password (401), unknown e-mail (404) and short passwords (400) with distinct messages. MongoDB stores users and analysis history when configured, otherwise an equivalent file store is used. The hosted static edition implements browser-local accounts with salted SHA-256 via WebCrypto so sign in / create account still work without a server, and resumes never leave the device.

3.6 Phase 5 - Interface Implementation

The UI was implemented component-first: Header with section highlighting, Hero, ExperienceCard (drag-drop, tabbed upload/paste, inline validation), RolesPanel results, searchable RolesPage with match badges and a role selector, RoleDetailPage with skill checklists and coverage bars, SignInPage (dual sign-in / sign-up modes), ProfilePage with saved history, and error/empty/loading states throughout. Styling uses the Tailwind design tokens of the brand palette; match percentages are deliberately session-scoped and never displayed before an analysis exists.

3.7 Phase 6 - Deployment and Packaging

In production a single Express process serves the built client and the API from one origin (eliminating cross-origin issues). The hosted edition is a separate static build with hash routing and an in-browser copy of the extraction and matching engines, published to GitHub Pages. The desktop edition embeds the server, client build and data file inside an Electron shell, compiled by electron-builder into a one-click NSIS installer published as a GitHub Release. Repository, releases and documentation are maintained on GitHub.

3.8 Phase 7 - Testing

Extraction tests: valid PDF, scanned PDF (OCR path), Word, plain text, wrong extension, >10 MB file, empty file.

Matching tests: alias spellings, title-hint roles, thin resumes (warm fallback), guaranteed minimum of six roles.

Auth tests: sign-up, duplicate e-mail, wrong password, unknown e-mail, short password, stale-session recovery.

Interface tests: percentages hidden until upload, resume-first sorting after upload, keyboard and mobile layouts.

Deployment tests: local full-stack, hosted static build and desktop installer verified end-to-end with the same resume.

3.9 Schedule of Work (12 weeks)

Weeks 1-2: requirement analysis, use-case definition, data modelling of nextrole.json.

Weeks 3-4: extraction pipeline - pdf-parse, pdf.js, OCR integration and error handling.

Weeks 5-6: matching engine, alias tables, scoring, Grok API adapter with fallback.

Weeks 7-8: authentication, profile persistence, REST API completion.

Weeks 9-10: full interface build, responsive styling, percentage-gating and UX fixes.

Week 11: testing across engines and browsers; packaging (Electron, installer) and hosting.

Week 12: documentation, synopsis, report and presentation preparation.

4. Facilities Required for the Proposed Work

4.1 Software Requirements

Operating system: Windows 10/11 (also runs on Linux and macOS).

Runtime: Node.js 20 or newer with npm; code editor: Visual Studio Code.

Languages and frameworks: TypeScript, React 18, Vite, Tailwind CSS, Express.

Database: MongoDB Community (optional - a file store is used when unavailable).

Resume engines: pdf-parse, pdf.js, Tesseract OCR with English language data, Mammoth.

Desktop packaging: Electron, electron-builder (NSIS installer).

Version control and hosting: Git, GitHub (repository, Releases, GitHub Pages).

Browsers for testing: Google Chrome / Microsoft Edge / Mozilla Firefox.

API testing: browser developer tools or curl; optional: xAI API key for Grok, Google Cloud OAuth client.

4.2 Hardware Requirements

Processor: x86-64 dual-core 2.0 GHz or better (quad-core recommended for OCR speed).

Memory: 4 GB RAM minimum, 8 GB recommended while running OCR and MongoDB together.

Storage: about 5 GB free disk space for toolchain, language data, builds and installer.

Network: broadband internet for GitHub, Google Fonts, optional Grok API and Google OAuth.

Display: 1366x768 or higher; no special peripherals are required.

5. Bibliography

1. Sommerville, I., Software Engineering, 10th Edition, Pearson Education, 2016. (Requirement analysis, feasibility and SDLC methods.)

2. Pressman, R. S. and Maxim, B. R., Software Engineering: A Practitioner's Approach, 8th Edition, McGraw-Hill, 2015. (Planning, testing and deployment process.)

3. Mozilla Developer Network (MDN) Web Docs - JavaScript, Fetch API and WebCrypto: https://developer.mozilla.org/

4. React Official Documentation, Meta Platforms Inc.: https://react.dev/

5. Node.js Official Documentation: https://nodejs.org/docs/latest/api/ ; Express.js Guide: https://expressjs.com/guide/

6. MongoDB Manual, MongoDB Inc.: https://www.mongodb.com/docs/manual/

7. Tailwind CSS Documentation: https://tailwindcss.com/docs ; Vite Guide: https://vitejs.dev/guide/

8. Mozilla pdf.js API Documentation: https://mozilla.github.io/pdf.js/api/ ; pdf-parse npm package: https://www.npmjs.com/package/pdf-parse

9. Tesseract.js - Optical Character Recognition in JavaScript: https://github.com/naptha/tesseract.js

10. Mammoth.js - Converting .docx to HTML/text: https://github.com/mwilliamson/mammoth.js

11. Electron Documentation, OpenJS Foundation: https://www.electronjs.org/docs ; electron-builder: https://www.electron.build/

12. xAI, Grok API Guide (chat-completions, models, authentication): https://docs.x.ai/

13. IETF RFC 7519 - JSON Web Token (JWT): https://www.rfc-editor.org/rfc/rfc7519

14. IETF RFC 7914 - The scrypt Password-Based Key Derivation Function: https://www.rfc-editor.org/rfc/rfc7914

15. Project source code, documentation, releases and this synopsis: https://github.com/saxenavansh070-droid/nextrole